DAY 2

Vibe Coding 实战课

前端 + Skill 机制

不是记语法，而是学会调动 AI 完成真实前端任务

AI 对话 · 前端实操 · Skill 沉淀

主讲：zane



DAY 1
认知与环境

DAY 2
前端 + Skill 机制

DAY 3
后端开发与模型接入

DAY 4
从 LLM 到 Agent - RAG

DAY 5
Git 协作与全栈部署

目录：今天会经历的 6 个阶段

每一章只解决一个问题，最后串成完整工作流。

01
先理解 VibeCoding

02
跑通前端最小工作流

03
完成个人主页和 Todo

04
理解 Claude Code 进阶能力

05
拆开 Skill 的结构

06
课后作业

课程节奏：先会用，再会改，最后会沉淀

每一段都服务于一个结果：独立启动并推进前端任务。



CLASS RULE

切记

工具会变，工作流会留下。你要记住的是如何描述问题、验证结果、把经验复用。

先抓结论	记住核心判断
再看例子	把概念放进真实前端场景里理解
马上应用	用一个动作把理解落到项目里

不背语法 不堆概念 / 重实操



TAKEAWAY

真正要学的是指挥 AI

AI 可以写很多代码，但方向、判断和验收仍然在你手里。

方向	
讲清结果	不要只说“帮我做”，要说给谁用、做到什么程度
补足现场	把项目状态、报错、约束告诉 AI

说清结果

先描述要交付的页面或功能

交代现场

补充项目状态、报错和已有文件

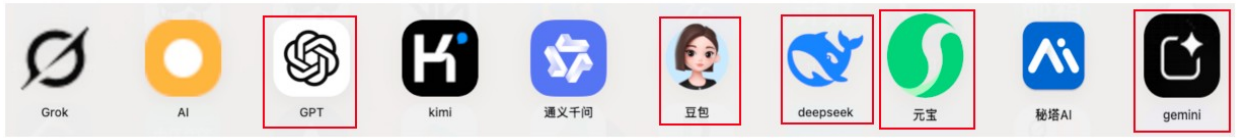
给出边界

说明不要改哪里、怎样检查完成

01

开场：为什么是 Vibe Coding

未来，一个人的力量取决于他能调动多少 AI 协同工作



1.1	不会就问 AI，是新的学习入口	07
1.2	在 VibeCoding 中，AI 不是老师，也不是搜索框	08
1.3	把 AI 当实习生	09
1.4	好问题，比好工具更重要	10
1.5	本节课最终产出	11
1.6	三小时路线图	12

1.1 不会就问 AI，是新的学习入口

遇到概念、报错、命令、代码结构，都可以先问 AI。关键不是问一次，而是追问到你能复述。

先定位	pwd 确认当前目录，ls 看有哪些文件
再执行	命令的作用范围通常由当前目录决定

开发判断

先定位	pwd 确认当前目录，ls 看有哪些文件
再执行	命令的作用范围通常由当前目录决定

```
pwd
ls
cd
mkdir
...
```

PowerShell 命令同理

学习入口 学习入口

1.2 在 VibeCoding 中，AI 不是老师，也不是搜索框

搜索框

- 给关键词
- 返回链接
- 你自己筛

开发搭档

- **给目标**
- **补上下文**
- **一起迭代**

判断标准 工具或概念没有绝对好坏，关键看任务、成本、上下文和验收方式。

1.3 把 AI 当实习生

实习生很勤快，但需要你交代清楚：做什么、为什么做、不能碰哪里、怎么验收。

目标	
先讲清结果	不要只说“帮我做”，要说给谁用、做到什么程度
补足现场	把项目状态、报错、约束告诉 AI

说清结果

先描述要交付的页面或功能

交代现场

补充项目状态、报错和已有文件

给出边界

说明不要改哪里、怎样检查完成

1.4 好问题，比好工具更重要

PROMPT

> 我想做一个个人主页。目标用户是招聘面试官。请先给我页面结构，不要直接写代码。

01 先说明要交付什么

02 明确不要大改、不要乱加依赖

03 最后要求 AI 给检查方式

落点 先要结构，再要代码，返工会少很多。

1.5 本节课最终产出

01

claude code 的使用

02

一个能交互的 Todo 网页

03

skill 美化过的 Todo 网页

04

一个可复用 Skill 雏形

1.6 三小时路线图

从会问 AI，到会沉淀 Skill。



关键原则 每一步都可以让 AI 参与，但每一步都要有人类验收。

02

前端最小地图

先知道自己正在改的是界面、逻辑，还是数据

2.1	前端是什么	14
2.2	后端是什么	15
2.3	前端和后端如何配合	16
2.4	前端不只是网页	17
2.5	前端架构	18
2.6	WebApp 是今天的主战场	19
2.7	WebApp 能做什么	20
2.8	前端开发有什么用	21
2.9	网页的三层结构	22
2.10	读 AI 写的代码，看这 4 件事	23
2.11	读代码不需要逐字读	24
2.12	浏览器 F12：前端开发者的显微镜	25

2.1 前端是什么

前端负责用户能看见、能点击、能感受到的部分。它把数据和功能变成可使用的界面。

01

看得见

页面内容、布局、按钮、表单都属于前端的表达范围

02

能操作

点击、输入、切换、提交，让用户能真正完成动作

03

会反馈

加载、成功、失败、空状态，决定体验是否可信

2.2 后端是什么

后端负责数据、权限、业务逻辑和服务稳定性。它让前端背后的系统真正运转。

管规则	登录权限、业务判断、数据校验都在后端完成
管数据	数据库、文件、日志和缓存让系统能长期运行

开发判断

管规则	登录权限、业务判断、数据校验都在后端完成
管数据	数据库、文件、日志和缓存让系统能长期运行

数据 数据 · 接口 · 业务

2.3 前端和后端如何配合



响应：后端返回数据，前端把数据变成页面

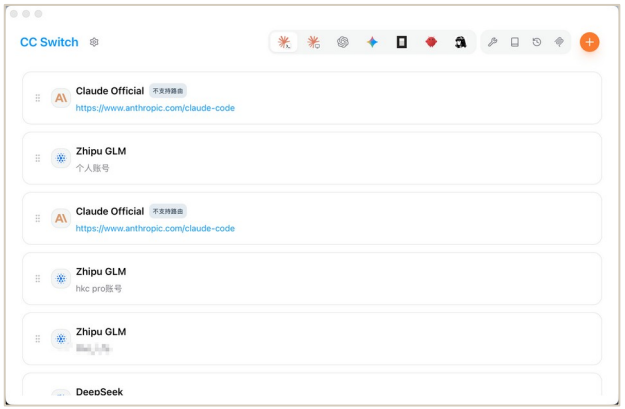
核心判断 前端不是“美化页面”，它要理解数据从哪里来、用户动作会触发什么请求。

2.4 前端不只是网页

客户端、WebApp、移动端、小程序，本质上都是把功能包装成用户能操作的界面。

多端



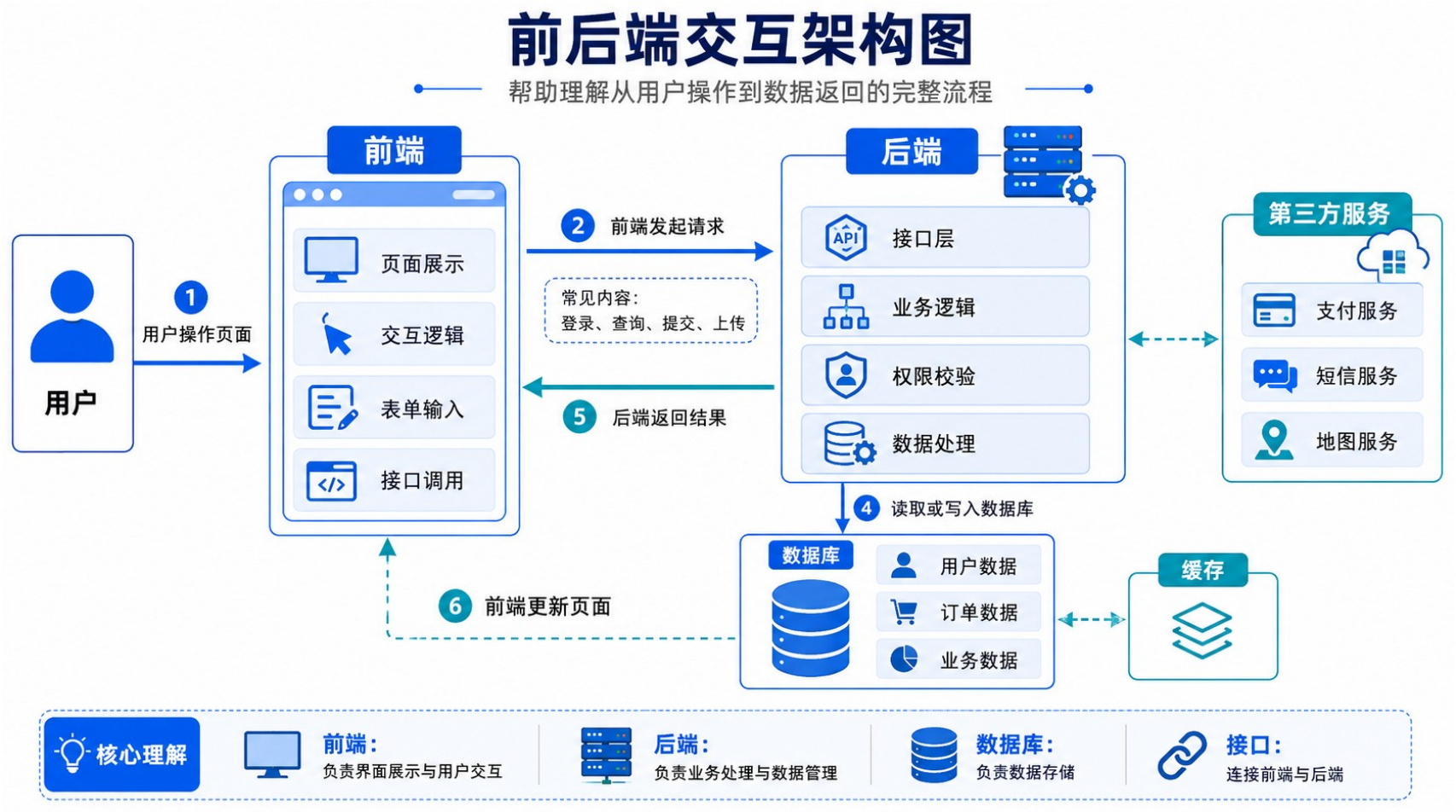






2.5 前端架构

前端是一组形态，不是一种单一页面。



2.6 WebApp 是今天的主战场

门槛低、反馈快、作品好展示。对初学者来说，WebApp 最适合用来练 Vibe Coding。

2.7 WebApp 能做什么

01

官网：把信息讲清楚

02

个人主页：展示你是谁

03

产品原型：验证一个想法

04

个人助手：把流程自动化

2.8 前端开发有什么用

前端是把想法变成可被看见、可被试用、可被传播的最快路径。





2.9 网页的三层结构

01

HTML

页面的骨架

02

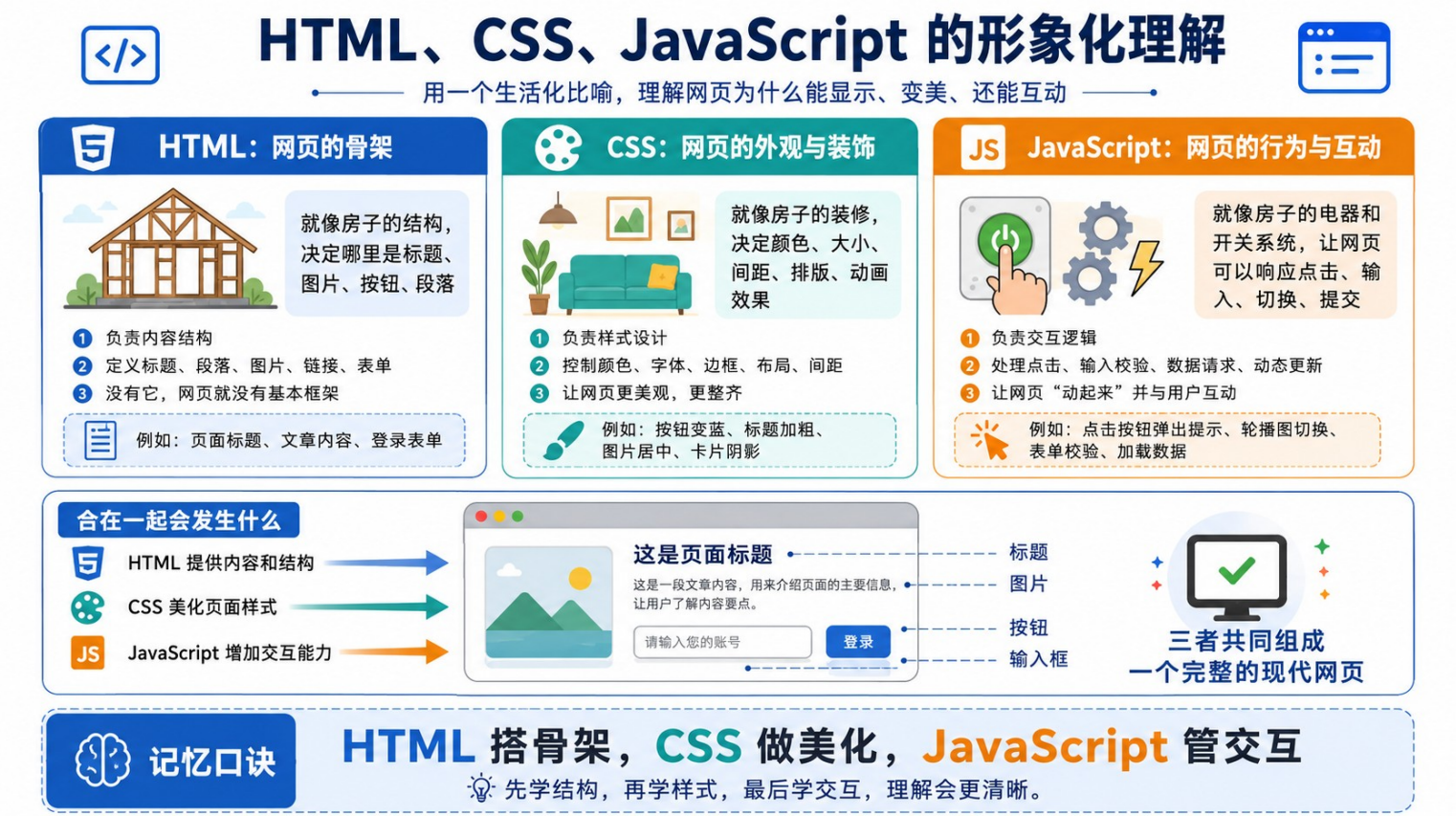
CSS

页面的样子

03

JavaScript

页面的行为



2.10 读 AI 写的代码，看这 4 件事

01

页面入口在哪里

02

组件怎么拆

03

样式放在哪里

04

用户操作触发了什么逻辑

2.11 读代码不需要逐字读

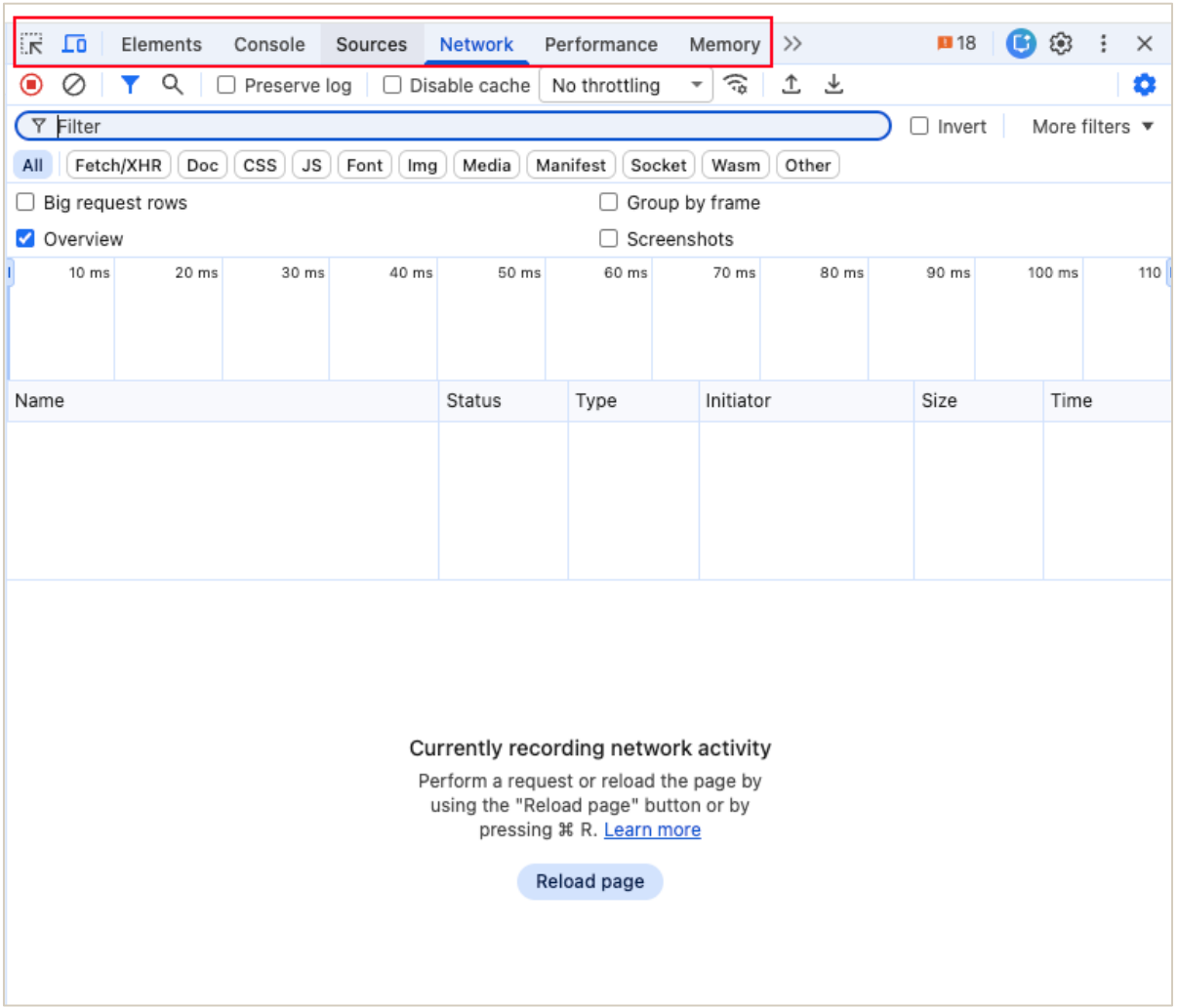
先看文件名，再看结构，再看关键函数。别一上来就陷进每一行语法。

先跑起来	能启动、能预览，才进入下一轮优化
再读结构	从 package.json、入口文件、组件目录开始看
小步修改	一次只改一个方向，避免大面积返工

先结构 先理解结构，再交给 AI 执行

2.12 浏览器 F12：前端开发者的显微镜

重点看三个区域：Elements 看结构，Console 看报错，Network 看请求。



03

Vibe Coding 工具链

工具不是目的，跑通工作流才是目的

3.1	IDE vs CLI：两种使用方式	27
3.2	今天会用到哪些工具	28
3.3	Claude Code：主力开发助手	29
3.4	Codex：另一种 AI Coding 工作流	30
3.5	Node 是什么	31
3.6	MacOS 上安装 Node	32
3.7	检查环境是否成功	33
3.8	终端不是黑魔法	34
3.9	最常用命令：看位置和文件	35
3.10	最常用命令：前端项目	36
3.11	Claude Code 安装检查	37
3.12	cc switch 是什么	38
3.13	实操：配置 DeepSeek API	39
3.14	实操：第一次 AI 对话	40

3.1 IDE vs CLI：两种使用方式

IDE

- 图形界面友好
- 适合看项目
- 适合初学者

CLI

- **命令更直接**
- **适合自动化**
- **适合深度开发**

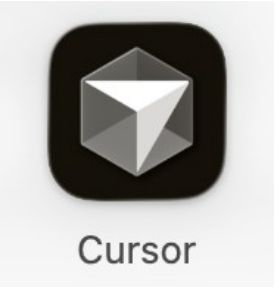
判断标准 工具或概念没有绝对好坏，关键看任务、成本、上下文和验收方式。

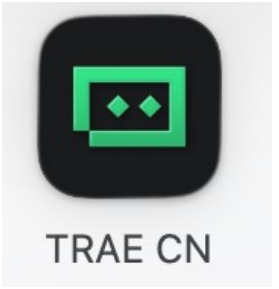
3.2 今天会用到哪些工具

Claude Code + VsCode/Cursor









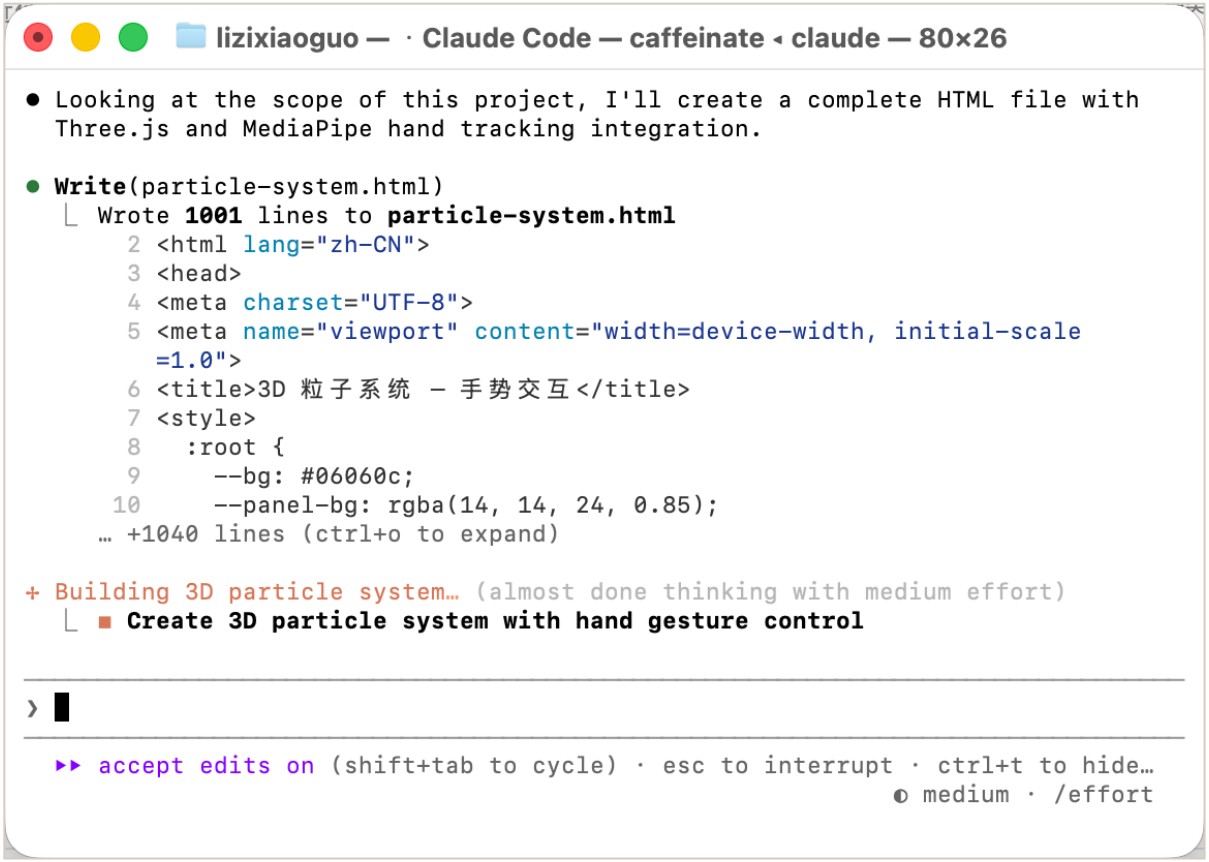


3.3 Claude Code：主力开发助手

适合在项目目录里对话、改代码、跑命令、持续修复。

项目级

定位	AI Coding 工具
适合	**项目内持续开发**
注意	生成结果必须本地运行验证

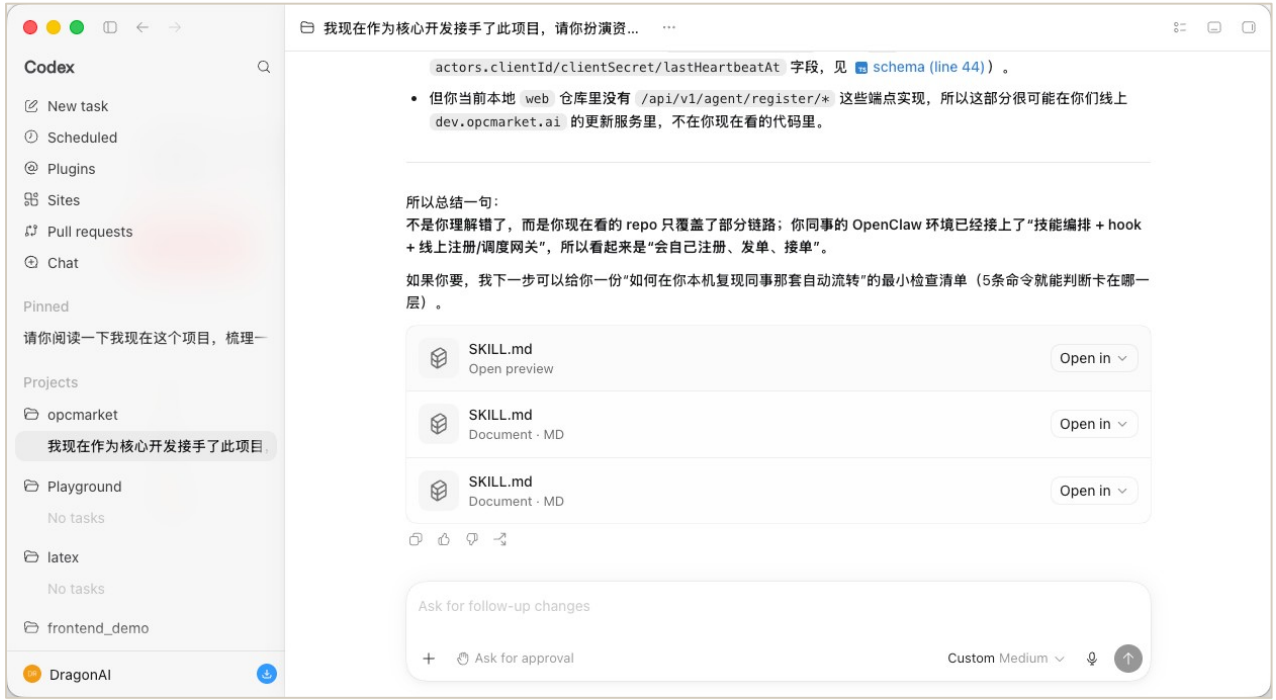


3.4 Codex：另一种 AI Coding 工作流

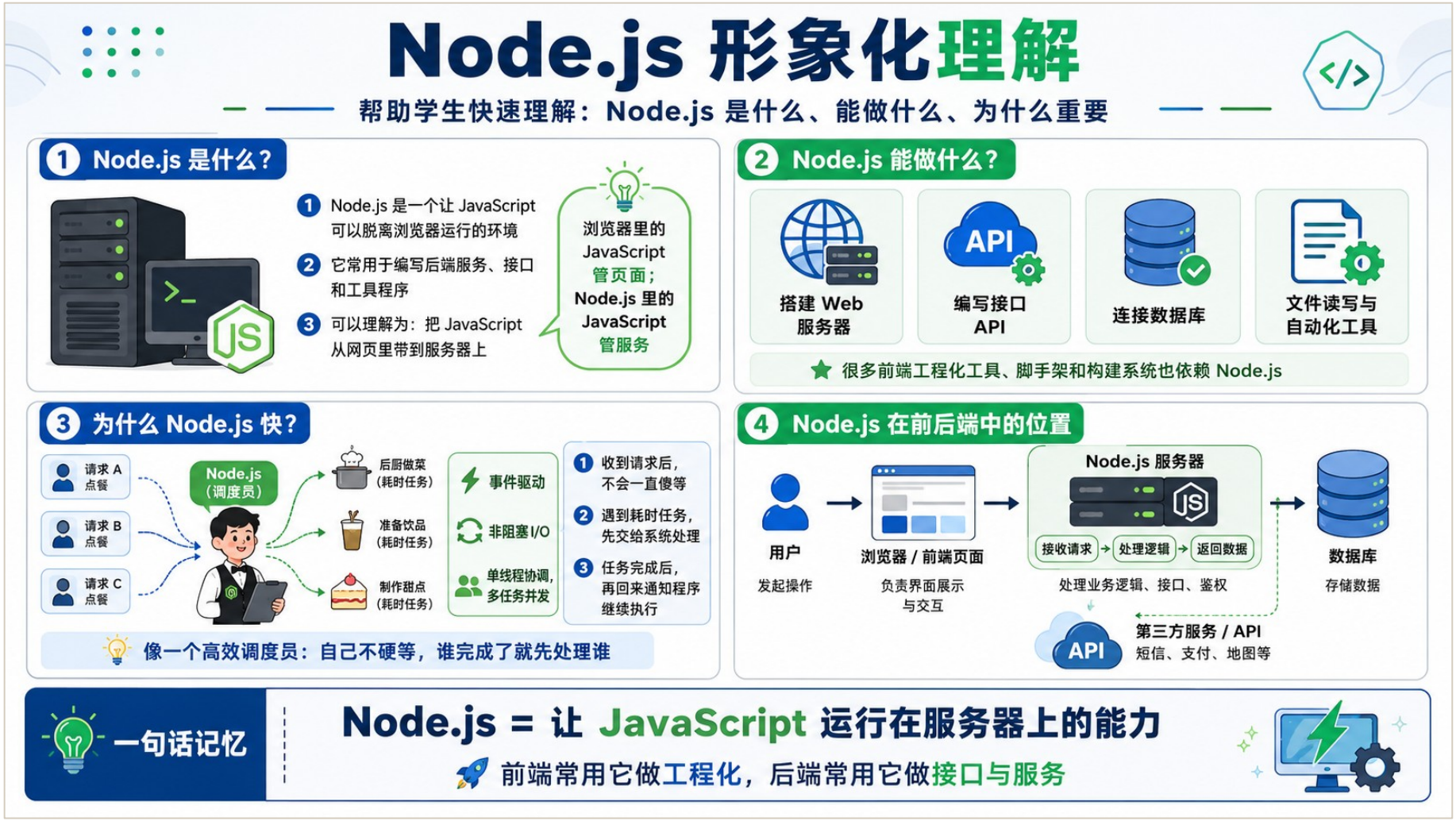
适合让 AI 读项目、做修改、产出可验证结果。它强调任务、修改和验证闭环。

对照

定位	AI Coding 工具
适合	**改代码、验证结果**
注意	生成结果必须本地运行验证



3.5 Node 是什么


Node.js 形象化理解
帮助学生快速理解：Node.js 是什么、能做什么、为什么重要
1 Node.js 是什么？
1 Node.js 是一个让 JavaScript 可以脱离浏览器运行的环境
2 它常用于编写后端服务、接口和工具程序
3 可以理解为：把 JavaScript 从网页里带到服务器上
浏览器里的 JavaScript 管页面；Node.js 里的 JavaScript 管服务
2 Node.js 能做什么？
搭建 Web 服务器
编写接口 API
连接数据库
文件读写与自动化工具
很多前端工程化工具、脚手架和构建系统也依赖 Node.js
3 为什么 Node.js 快？
请求 A 点餐
请求 B 点餐
请求 C 点餐
Node.js（调度员）
后厨做菜（耗时任务）
准备饮品（耗时任务）
制作甜点（耗时任务）
事件驱动
非阻塞 I/O
单线程协调，多任务并发
1 收到请求后，不会一直傻等
2 遇到耗时任务，先交给系统处理
3 任务完成后，再回来通知程序继续执行
像一个高效调度员：自己不硬等，谁完成了就先处理谁
4 Node.js 在前后端中的位置
用户
发起操作
浏览器 / 前端页面
负责界面展示与交互
Node.js 服务器
接收请求
处理逻辑
返回数据
处理业务逻辑、接口、鉴权
数据库
存储数据
API
第三方服务 / API
短信、支付、地图等
一句话记忆
Node.js = 让 JavaScript 运行在服务器上的能力
前端常用它做工程化，后端常用它做接口与服务

3.6 MacOS 上安装 Node

Mac

```
$ brew install node
$ node -v
$ npm -v
```

3.7 检查环境是否成功

```
$ which node
$ which npm
$ npm create vite@latest
```

记住 检查比安装更重要：你要知道当前命令到底指向哪里。

3.8 终端不是黑魔法

终端只是用文字控制电脑。你不需要记住全部命令，只要知道怎么问 AI。

01

先定位

pwd 确认当前目录，ls 看有哪些文件

02

再执行

命令的作用范围通常由当前目录决定

03

看输出

终端日志比最后一行更重要

3.9 最常用命令：看位置和文件

```
$ pwd
$ ls
$ cd your-project
```

01 先确认命令在哪个目录执行

02 执行后看完整输出

03 报错连同命令一起交给 AI

记住 先确认自己在哪，再执行下一步。

3.10 最常用命令：前端项目

```
$ npm install
$ npm run dev
$ npm run build
```

01 先确认命令在哪个目录执行

02 执行后看完整输出

03 报错连同命令一起交给 AI

记住 跑不起来时，把完整报错交给 AI。

3.11 Claude Code 安装检查

```
$ claude --version
$ claude
$ /help
```

01 先确认命令在哪个目录执行

02 执行后看完整输出

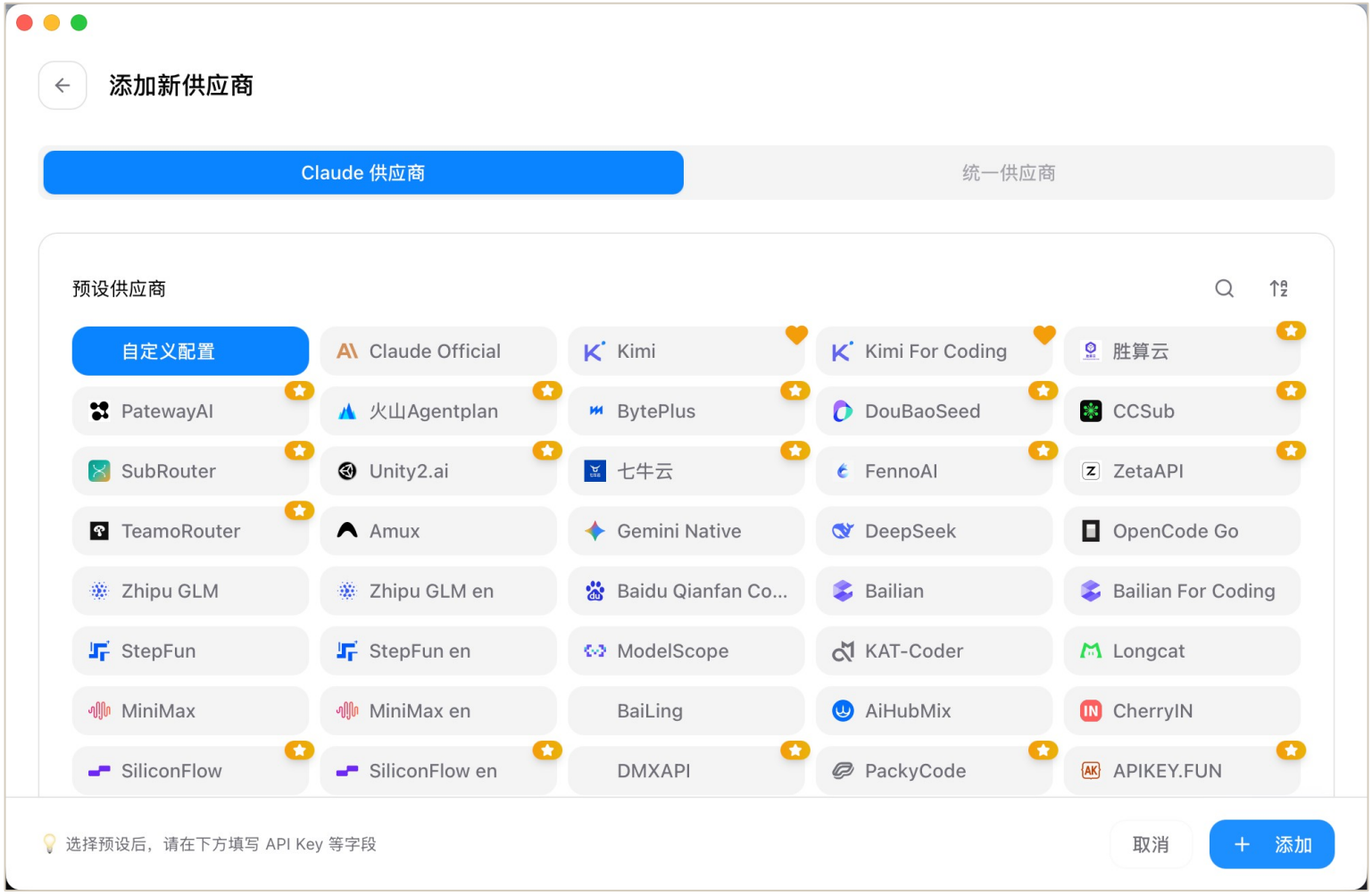
03 报错连同命令一起交给 AI

windows 用户请用 PowerShell

记住 能打开对话，不代表项目环境已经配置好。

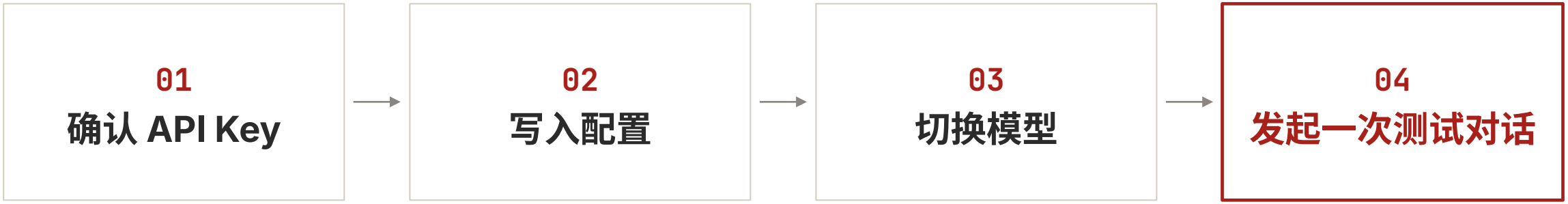
3.12 cc switch 是什么

它帮助你在不同模型和 API 配置之间切换，减少手动改配置的混乱。



3.13 实操：配置 DeepSeek API

让 Claude Code 能通过 cc switch 使用 DeepSeek。

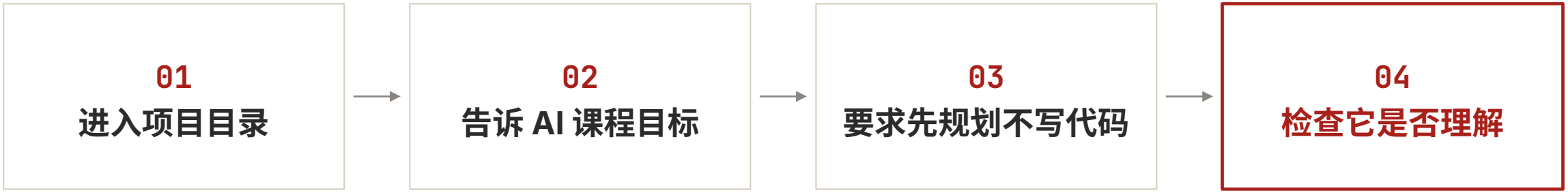


先跑通完整链路，再追求好看。

产出 能得到一次稳定回复

3.14 实操：第一次 AI 对话

让 AI 解释当前项目目录，并给出下一步建议。



先跑通完整链路，再追求好看。

产出 一份可执行计划

04

和 AI 对话的标准姿势

你说得越清楚，AI 返工越少

4.1	一个坏 Prompt 长什么样	42
4.2	一个好 Prompt 长什么样	43
4.3	遇到不会的问题，怎么问	44
4.4	遇到报错，怎么问	45
4.5	让 AI 先规划，再执行	46
4.6	提示词不是玄学，是工作流	47

4.1 一个坏 Prompt 长什么样

坏

- 帮我做个网页
- 越酷越好
- 直接给代码

问题

- **目标不清**
- **风格不清**
- **没有验收**

判断标准 工具或概念没有绝对好坏，关键看任务、成本、上下文和验收方式。

4.2 一个好 Prompt 长什么样

角色设定：你是一位专注于 SaaS 产品和高端个人品牌的 UI/UX 设计师。

任务目标：在不改变现有页面结构和内容的前提下，仅从 **Visual/UI** 角度对我规划的"个人招聘主页"进行美化升级。

当前状态：页面包含简介、项目、技能、联系方式四个板块，目前只有基础的 HTML/CSS 结构。

美化要求（Design Guidelines）：

1. **风格定义**：**Minimalist & Professional（极简专业风）**。参考 Linear、Vercel 或 Apple 官网的设计语言。拒绝"五彩斑斓的黑"，追求克制的高级感。
2. **排版（Typography）**： 制定清晰的字体层级（H1/H2/H3/正文）。 推荐使用系统原生字体栈（San Francisco, Inter, Segoe UI），确保跨平台渲染一致。 强调行高（Leading）和字间距（Tracking）的调整，增加呼吸感。
3. **色彩系统**： 背景色使用柔和的 Off-white（如 `#F9FAFB`）代替纯白，减轻视觉疲劳。 文字使用深灰（如 `#1F2937`）代替纯黑，柔和护眼。 强调色（Accent Color）： 选定一种主色调（如 Indigo 或 Teal）用于链接和按钮悬停效果。
4. **空间与布局（Spacing & Layout）**： 采用"大量留白"策略，内容区域居中，最大宽度限制在 `max-w-3xl` 左右，避免长文本阅读压力。 利用 Grid 或 Flexbox 对齐项目卡片，确保视觉平衡。
5. **细节打磨（Micro-interactions）**： 为链接添加优雅的下划线动画（hover时从中间向两边展开）。 为项目卡片添加极其细微的阴影（Shadow-sm）和鼠标悬停时的上浮效果（Hover lift）。 技能标签（Tags）使用浅色背景+深色文字，圆角处理，营造"胶囊"感。
6. **质感**：使用 1px 的细边框（Border neutral-200）来区分区域，增加精致度。

输出要求：

- 请描述视觉优化的具体方向和细节（例如："将项目卡片的背景改为透明，仅保留 1px 边框，Hover 时填充极淡的灰色"）。
- **不要写代码**，只需描述视觉效果和设计逻辑。
- 告诉我这种设计方案如何提升 HR 的阅读体验。

4.3 遇到不会的问题，怎么问

PROMPT

> 我不懂这个概念。请先用大二学生能理解的话解释，再给一个前端里的例子，最后告诉我是否需要现在掌握。

落点 把解释粒度说清楚。

4.4 遇到报错，怎么问

PROMPT

> 这是完整报错和我刚执行的命令。请先判断错误类型，再列出 3 个可能原因，最后只做最小修改。

落点 报错要连同命令和上下文一起给。

4.5 让 AI 先规划，再执行

复杂任务先进入规划模式。你确认方案后，再让 AI 动手写代码。

01

先讲清结果

不要只说“帮我做”，要说给谁用、做到什么程度

02

补足现场

把项目状态、报错、约束告诉 AI

03

要求解释

看不懂改动时，先让 AI 解释再执行

4.6 提示词不是玄学，是工作流

每一次高质量对话，都在压缩下一次返工。



关键原则 每一步都可以让 AI 参与，但每一步都要有人类验收。

05

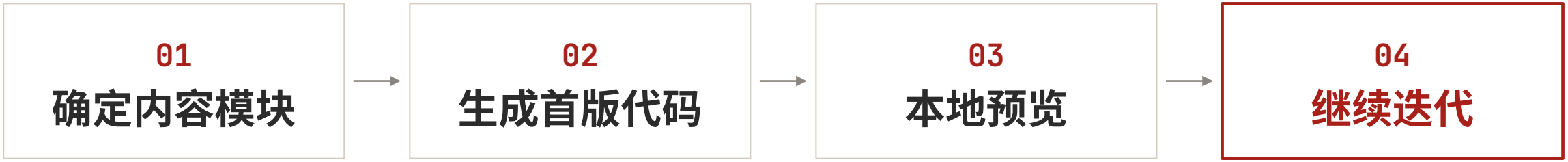
第一次前端实操：个人主页

先做一个能打开、能展示、能继续改的作品

5.1	实操目标：做一个个人主页	49
5.2	第一步：描述你想要的页面	50
5.3	第二步：让 AI 生成代码	51
5.4	第三步：打开预览	52
5.5	第四步：用 F12 检查问题	53
5.6	第五步：继续对话修改	54
5.7	页面好不好，不只看能不能打开	55
5.8	让 AI 帮你解释代码结构	56
5.9	个人主页小结	57

5.1 实操目标：做一个个人主页

面向招聘或作品展示，做出一个可打开的单页。



先跑通完整链路，再追求好看。

产出 个人主页 Demo

5.2 第一步：描述你想要的页面

PROMPT

> 请为我设计一个个人主页，目标是展示前端学习能力。页面包括 Hero、技能、项目、联系方式。先给结构和文案。

落点 先让 AI 想清楚页面，不急着写代码。

5.3 第二步：让 AI 生成代码

告诉 AI 技术限制：先用 HTML、CSS、JS，不引入复杂框架。

拆组件	把页面拆成可维护的按钮、卡片、列表和表单
管状态	让数据变化能稳定反映到页面上

最小可用 最小可用

5.4 第三步：打开预览

```
$ open index.html
$ 或者使用 Live Server
$ 或者启动 dev server
```

记住 能看到页面，是第一轮验收。

5.5 第四步：用 F12 检查问题

页面能打开只是第一步。Console 没有报错、布局没有破版，才算通过第一轮验收。

配图原则 只补一张关键截图，清楚显示工具或页面状态。

5.6 第五步：继续对话修改

PROMPT

> 页面已经能打开。请帮我改善层级和间距，不要增加新功能。修改前先说明你要改哪几个区域。

落点 一次只改一个方向。

5.7 页面好不好，不只看能不能打开

01
标题是否清楚

02
模块是否完整

03
移动端是否可看

04
按钮是否可点击

05
内容是否像真实作品

检查方式 清单不是用来背的，是用来判断 AI 输出到底能不能交付。

5.8 让 AI 帮你解释代码结构

不要只拿结果。让 AI 解释每个文件、每个模块、关键样式为什么这样写。

先讲清结果	不要只说“帮我做”，要说给谁用、做到什么程度
补足现场	把项目状态、报错、约束告诉 AI
要求解释	看不懂改动时，先让 AI 解释再执行

反推理解 先理解结构，再交给 AI 执行

5.9 个人主页小结

你已经完成了第一个前端作品。下一步不是换工具，而是把它变成项目。

第一个作品	
先跑起来	能启动、能预览，才进入下一轮优化
再读结构	从 package.json、入口文件、组件目录开始看

入口

先找 main / App / package.json

结构

组件、样式、工具函数分别放哪

风险

不要一次让 AI 大面积重写

06

前端框架入门

框架的意义，是让页面能被长期维护

6.1	为什么需要前端框架	59
6.2	组件化：把页面拆成零件	60
6.3	Vue、React、Next.js 的区别	61
6.4	Vue3：适合入门理解组件	62
6.5	实操：初始化 Vue3 项目	63
6.6	React：主流产品开发生态	64
6.7	实操：初始化 React 项目	65
6.8	Next.js：从页面走向产品	66
6.9	AI 写项目时，你重点看这些文件	67
6.10	让 AI 解释项目目录	68

6.1 为什么需要前端框架

当页面变多、交互变复杂、状态变多，原生写法会越来越难维护。

拆组件	把页面拆成可维护的按钮、卡片、列表和表单
管状态	让数据变化能稳定反映到页面上

开发判断

拆组件	把页面拆成可维护的按钮、卡片、列表和表单
管状态	让数据变化能稳定反映到页面上

复杂度 复杂度

6.2 组件化：把页面拆成零件

导航、按钮、卡片、列表、表单，都可以变成可复用组件。

先抓结论	这一页只记住一个核心判断
再看例子	把概念放进真实前端场景里理解
马上应用	用一个动作把理解落到项目里

组件 先理解结构，再交给 AI 执行

6.3 Vue、React、Next.js 的区别

Vue / React

- 更偏前端组件
- 适合学习交互
- 生态成熟

Next.js

- **页面和路由更完整**
- **接近产品开发**
- **能做全栈能力**

```
Vite npm/pnpm react/vue
```

判断标准 工具或概念没有绝对好坏，关键看任务、成本、上下文和验收方式。

6.4 Vue3：适合入门理解组件

模板直观，学习曲线相对平滑，适合初学者理解数据和页面如何绑定。

Vue3

定位	渐进式前端框架
适合	**项目内持续开发**
注意	生成结果必须本地运行验证



6.5 实操：初始化 Vue3 项目

```
$ npm create vite@latest vue-demo
$ cd vue-demo
$ npm install
$ npm run dev
```

01 先确认命令在哪个目录执行

02 执行后看完整输出

03 报错连同命令一起交给 AI

记住 不要急着改，先跑起来。

6.6 React：主流产品开发生态

React 更强调用 JavaScript 描述界面，是很多公司前端项目的主流选择。

React

定位	组件化 UI 生态
适合	**项目内持续开发**
注意	生成结果必须本地运行验证



6.7 实操：初始化 React 项目

```
$ npm create vite@latest react-demo
$ cd react-demo
$ npm install
$ npm run dev
```

01 先确认命令在哪个目录执行

02 执行后看完整输出

03 报错连同命令一起交给 AI

记住 让 AI 解释 src 目录里的入口文件。

6.8 Next.js：从页面走向产品

Next.js 不只是 UI，它包含路由、渲染、接口等更接近产品的能力。

Next.js

定位	产品级 React 框架
适合	**项目内持续开发**
注意	能力更强，概念也更多



6.9 AI 写项目时，你重点看这些文件

01

package.json

02

src/main 或 src/App

03

components 目录

04

styles 或 CSS 文件

05

README

检查方式 清单不是用来背的，是用来判断 AI 输出到底能不能交付。

6.10 让 AI 解释项目目录

PROMPT

> 请用表格解释这个前端项目的目录结构。告诉我哪些文件是入口，哪些文件可以先不看。

01 先说明要交付什么

02 明确不要大改、不要乱加依赖

03 最后要求 AI 给检查方式

落点 目录理解，是接手项目的第一步。

07

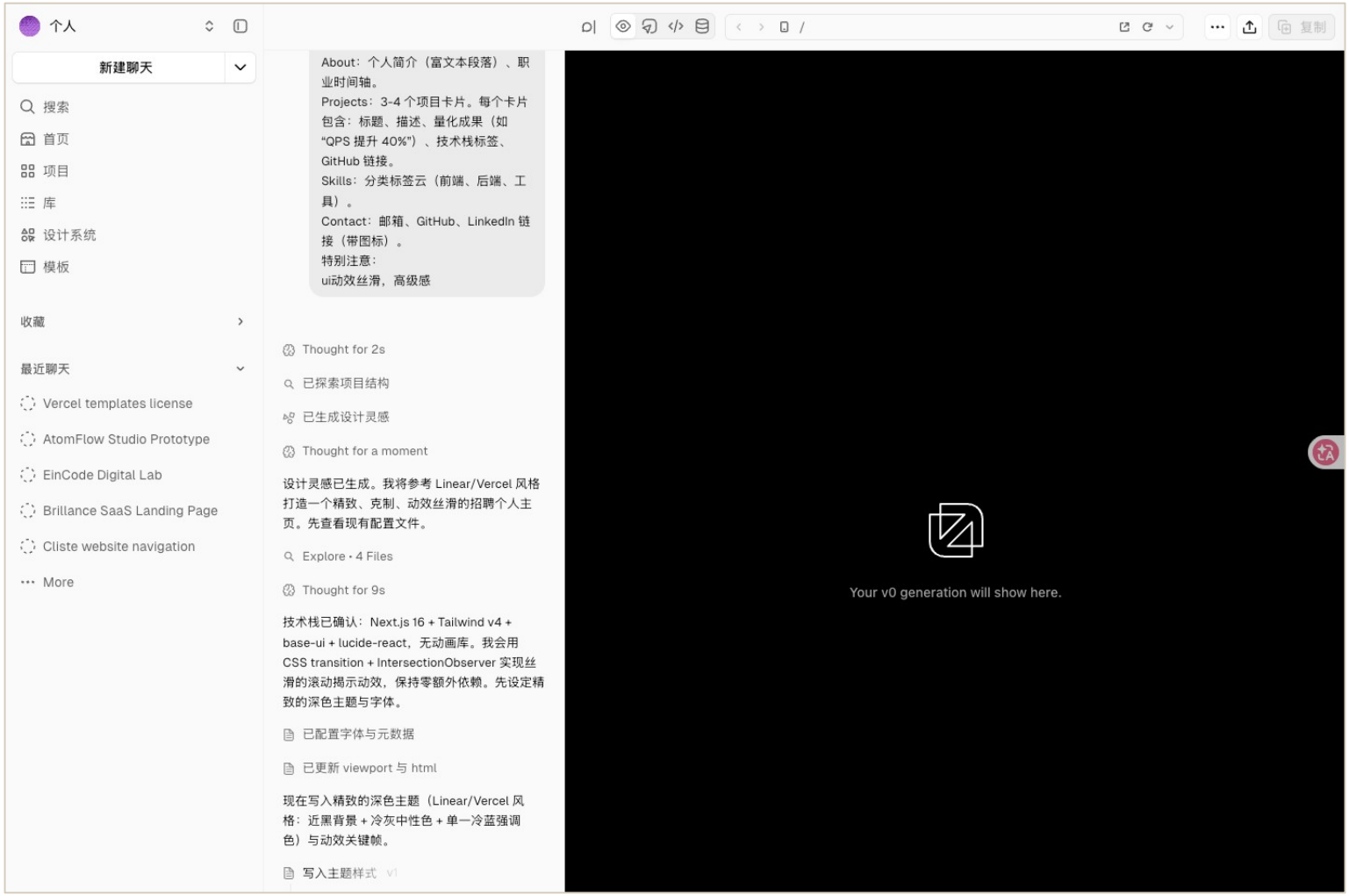
v0.dev 与 Todo 项目

界面可以生成，但工程仍要你接住

7.1	v0.dev 适合做什么	70
7.2	好页面 Prompt 的四个要素	71
7.3	实操：用 v0.dev 生成个人主页	72
7.4	第一个完整项目：Todo 网页	73
7.5	Todo 迭代路线	74
7.6	版本回滚：大胆试错的底气	75

7.1 v0.dev 适合做什么

它适合生成界面初稿：Prompt 输入、页面预览、代码输出形成一个快速原型闭环。



7.2 好页面 Prompt 的四个要素

01

页面类型

个人主页 / 官网 / 后台

02

视觉风格

简洁 / 科技 / 课程

03

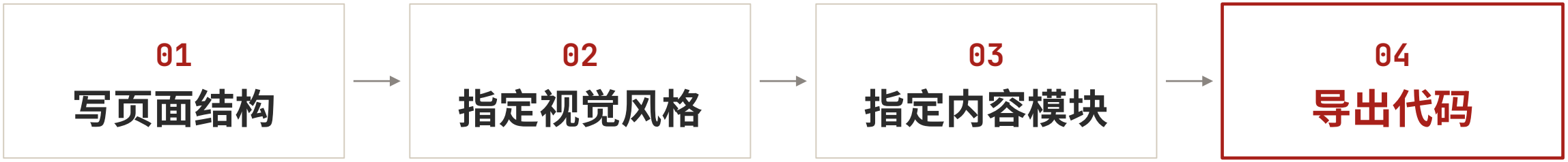
内容结构

模块和优先级

记法 先给三者定位，再用一个页面例子串起来，概念就不会散。

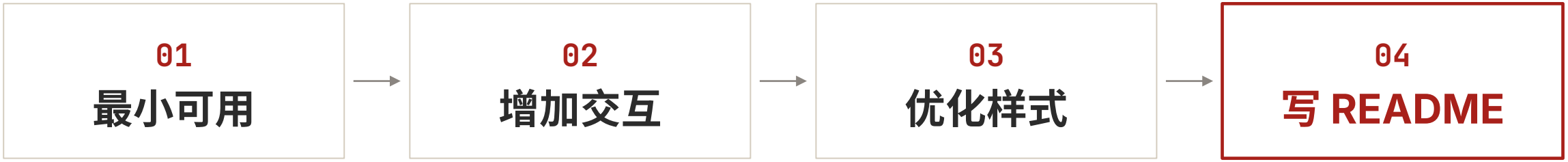
7.3 实操：用 v0.dev 生成个人主页

用更好的 Prompt 生成一个页面版本。



7.4 第一个完整项目：Todo 网页

做一个能新增、完成、删除任务的 Todo。



产出 Todo Demo

7.5 Todo 迭代路线

不要一次做完，分四轮让 AI 逐步推进。



关键原则 每一步都可以让 AI 参与，但每一步都要有人类验收。

7.6 版本回滚：大胆试错的底气

每一次大改前都保存版本。做错不可怕，回不去才可怕。

先抓结论	这一页只记住一个核心判断
再看例子	把概念放进真实前端场景里理解

开发判断

先抓结论	这一页只记住一个核心判断
再看例子	把概念放进真实前端场景里理解

可回滚 可回滚

08

Claude Code 进阶工作流

从一次对话，变成可控的开发过程

8.1	三种模式：对话、规划、执行	77
8.2	什么时候用规划模式	78
8.3	什么时候让 AI 执行终端命令	79
8.4	后台任务管理	80
8.5	上下文压缩与清除	81
8.6	项目记忆文件	82
8.7	Hook：把固定动作自动化	83
8.8	Sub Agent：把复杂任务拆出去	84

8.1 三种模式：对话、规划、执行

模式	提示	你要关注
默认	? for shortcuts	修改文件前一定询问用户
规划	plan mode on	只讨论、不修改文件
自动	accepts edits on	自动修改文件

8.2 什么时候用规划模式

跨多个文件、影响比较大的任务，先规划，再执行。

01

先抓结论

这一页只记住一个核心判断

02

再看例子

把概念放进真实前端场景里理解

03

马上应用

用一个动作把理解落到项目里

8.3 什么时候让 AI 执行终端命令

安装依赖、启动项目、运行测试、查看日志，这些都适合交给 AI 执行。

先定位	pwd 确认当前目录，ls 看有哪些文件
再执行	命令的作用范围通常由当前目录决定

开发判断

先定位	pwd 确认当前目录，ls 看有哪些文件
再执行	命令的作用范围通常由当前目录决定

命令 命令

8.4 后台任务管理

开发服务器会持续运行。你要知道它是否启动成功、端口是多少、如何停止。

先抓结论	这一页只记住一个核心判断
再看例子	把概念放进真实前端场景里理解
马上应用	用一个动作把理解落到项目里

`dev server` 先理解结构，再交给 AI 执行

8.5 上下文压缩与清除

对话太长会影响判断。阶段性总结，让 AI 保留关键上下文。

上下文

先讲清结果	不要只说“帮我做”，要说给谁用、做到什么程度
补足现场	把项目状态、报错、约束告诉 AI

说清结果

先描述要交付的页面或功能

交代现场

补充项目状态、报错和已有文件

给出边界

说明不要改哪里、怎样检查完成

8.6 项目记忆文件

把项目规则、命令、目录说明写下来，让 AI 每次进项目都先读。

01

先定位

pwd 确认当前目录，ls 看有哪些文件

02

再执行

命令的作用范围通常由当前目录决定

03

看输出

终端日志比最后一行更重要

8.7 Hook：把固定动作自动化

例如改完代码后自动格式化、自动运行检查、自动提醒风险。

先跑起来	能启动、能预览，才进入下一轮优化
再读结构	从 package.json、入口文件、组件目录开始看

开发判断

先跑起来	能启动、能预览，才进入下一轮优化
再读结构	从 package.json、入口文件、组件目录开始看

自动化 自动化

8.8 Sub Agent：把复杂任务拆出去

主对话负责决策，子任务负责搜索、检查、生成局部方案。

先抓结论	这一页只记住一个核心判断
再看例子	把概念放进真实前端场景里理解
马上应用	用一个动作把理解落到项目里

分工 先理解结构，再交给 AI 执行

09

Skill 机制

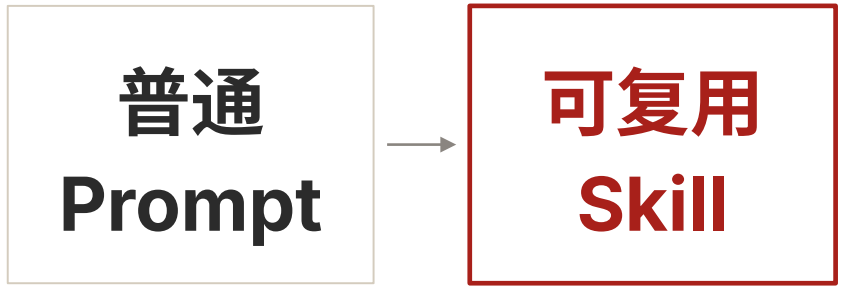
把一次做对，变成以后稳定做对

9.1	Skill 是什么	86
9.2	为什么需要 Skill	87
9.3	Skill 的本质	88
9.4	Skill 的目录结构	89
9.5	Skill 里面通常有什么	90
9.6	Skill 和 Prompt 的区别	91
9.7	Skill 和 Sub Agent 的区别	92
9.8	Plugin、Skill、工具分别解决什么	93
9.9	前端开发 Skill 可以做什么	94
9.10	实操：拆解一个具体 Skill	95
9.11	实操：用 Skill 改进 Todo 网页	96

9.1 Skill 是什么

Skill 是写给 AI 的“专业工作说明书”。

触发条件	什么时候应该使用这套方法
执行流程	先做什么、后做什么、哪里要谨慎
质量标准	输出做到什么程度才算合格



9.2 为什么需要 Skill

同一件事反复解释，是 AI Coding 里最隐形的浪费。

没有 Skill	每次重新说明风格、流程、限制	**容易漏、容易偏、容易返工**
有 Skill	规则被固定在文件里，AI 先读再做	**同类任务输出更稳定**

核心 Skill 不是让 AI 更聪明，而是让它更稳定地按你的标准做事。

9.3 Skill 的本质

Skill 不是知识库，而是可复用上下文。

它把“怎么做”从聊天记录里拿出来，放进项目能力里。

不是

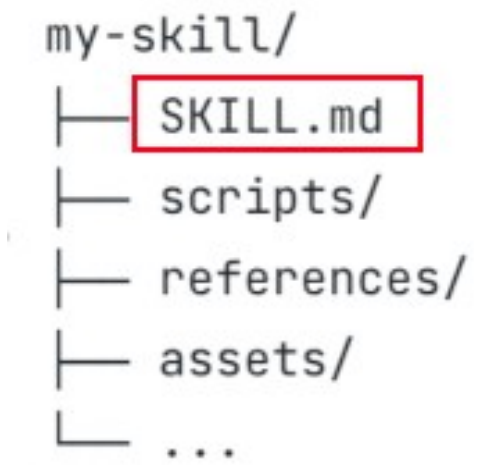
- 一次性灵感
- 一句万能咒语
- 替代你的判断

而是

- 稳定流程
- 质量标准
- 项目经验沉淀

9.4 Skill 的目录结构

一个 Skill 通常由说明文件、参考资料、脚本和素材组成，核心入口是 SKILL.md。



配图原则 只补一张关键截图，清楚显示工具或页面状态。

9.5 Skill 里面通常有什么

01
什么时候使用

02
工作步骤

03
质量标准

04
示例输入输出

05
可复用脚本或素材

检查方式 清单不是用来背的，是用来判断 AI 输出到底能不能交付。

9.6 Skill 和 Prompt 的区别

Prompt

- 一次性
- 适合当前问题
- 容易散落

Skill

- **可复用**
- **适合一类任务**
- **能持续改进**

判断标准 工具或概念没有绝对好坏，关键看任务、成本、上下文和验收方式。

9.7 Skill 和 Sub Agent 的区别

Skill

- 定义做事方法
- 提升稳定性
- 像操作手册

Sub Agent

- **拆出去做任务**
- **提升并行度**
- **像临时助手**

判断标准 工具或概念没有绝对好坏，关键看任务、成本、上下文和验收方式。

9.8 Plugin、Skill、工具分别解决什么

把三者分清楚，后面接入生态就不会乱。

工具	**提供能力**	能搜索、能读文件、能生成图片、能跑命令
Skill	**提供方法**	告诉 AI 遇到某类任务时怎么做才稳
Plugin	**打包生态**	把工具、Skill、配置组合成可安装能力

9.9 前端开发 Skill 可以做什么

01

统一 UI 风格

02

检查移动端适配

03

生成组件规范

04

修复常见报错

9.10 实操：拆解一个具体 Skill

看懂 Skill 如何触发、如何约束输出、如何验证结果。

```
https://getdesign.md/apple/design-md
```

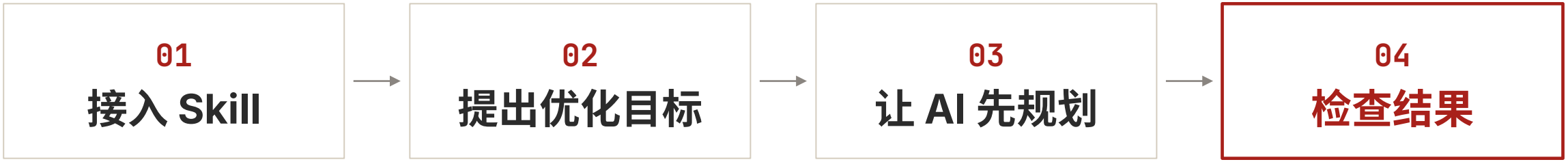
- impeccable
- taste skill
- shadcn
- ui-ux-pro-max
- design MD Collection

网页动效组件

originkit
react bits
unicorn

9.11 实操：用 Skill 改进 Todo 网页

让 AI 按固定前端规范优化项目。



先跑通完整链路，再追求好看。

产出 更稳定的 Todo 项目

10

总结与课后练习

把每一次开发，都变成下一次的起点

10.1	AI 很快，但你必须会验收	98
10.2	不要踩这些坑	99
10.3	今天真正学到的是一条工作流	100
10.4	课后练习：智能客服官网	101
10.5	作业要求：页面结构	102
10.6	作业要求：交互功能	103
10.7	作业要求：交付物	104
10.8	从会用 AI，到会指挥 AI	105

10.1 AI 很快，但你必须会验收

速度不是质量。你要看页面、跑项目、读报错、检查文件变化。

01

先讲清结果

不要只说“帮我做”，要说给谁用、做到什么程度

02

补足现场

把项目状态、报错、约束告诉 AI

03

要求解释

看不懂改动时，先让 AI 解释再执行

10.2 不要踩这些坑

01

不要复制自己看不懂的代码

02

不要一次让 AI 做太多事

03

不要忽略报错

04

不要没有版本回滚

05

不要把生成当完成

检查方式 清单不是用来背的，是用来判断 AI 输出到底能不能交付。

10.3 今天真正学到的是一条工作流

描述任务 → 生成代码 → 本地预览 → 定位问题 → 继续迭代 → 沉淀 Skill。

先跑起来	能启动、能预览，才进入下一轮优化
再读结构	从 package.json、入口文件、组件目录开始看
小步修改	一次只改一个方向，避免大面积返工

10.4 课后练习：智能客服官网

目标是做出一个能展示产品价值、能引导咨询、能在手机上正常浏览的官网。

10.5 作业要求：页面结构

01
首页 Hero

02
产品能力介绍

03
适用场景

04
价格或套餐

05
联系方式

检查方式 清单不是用来背的，是用来判断 AI 输出到底能不能交付。

10.6 作业要求：交互功能

01

导航跳转

02

表单输入

03

FAQ 展开收起

04

移动端适配

检查方式 清单不是用来背的，是用来判断 AI 输出到底能不能交付。

10.7 作业要求：交付物

01

可运行项目

02

README

03

截图

04

一段使用 AI 的过程记录

检查方式 清单不是用来背的，是用来判断 AI 输出到底能不能交付。

10.8 从会用 AI，到会指挥 AI

真正的变化不是 AI 帮你写了多少代码，而是你开始用工程方法管理 AI 的输出。

提示 这就是 Vibe Coding 的核心。

把每一次开发，
都变成下一次的 Skill

CLOSING
谢谢